

PYGAME, FROM SCRATCH

Just the loop. Just the window.

No physics today. No SMD. Slow down and learn what pygame actually is before stacking anything on top.

What pygame is (and isn't).

pygame is a thin library that opens a window and lets you color pixels in it. That's the whole story. Everything else — sprites, sound, physics — is just patterns built on top of those two things.

What it gives you

- A window that your OS draws on the screen.
- A function to fill that window with color.
- Functions to draw rectangles, circles, lines, images.
- A queue of input events (keys, mouse, window-close).
- A clock that lets you control frame rate.

What it does NOT give you

- Physics — you compute positions yourself.
- Collision detection beyond rectangle-overlap.
- Scene management — you build that yourself.
- A renderer — drawing is immediate, pixel-level.
- An entity system — sprites help but you wire them up.



Today's goal: get a window open, draw one thing in it, control frame rate.

That's it. We earn complexity tomorrow.

The shortest pygame program.

Eight lines that open a window, keep it open until you click the X, and clean up after itself. Memorize the shape of this — every pygame program starts here.

```
import pygame

pygame.init()                # wake up pygame
screen = pygame.display.set_mode((640, 480)) # ask OS for a window
pygame.display.set_caption("My first pygame")

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
    pygame.display.flip()      # show whatever we drew

pygame.quit()
```

Line by line

- `init()` — boots pygame's subsystems (display, audio, fonts).
- `set_mode((w,h))` — asks the OS for a window. Returns a Surface (the screen).
- `set_caption` — the title-bar text. Optional but tidy.
- `while running` — the heartbeat. Without it, window flashes for milliseconds and dies.
- `event.get()` — collect everything since the last call. We only check for QUIT.
- `display.flip()` — push your drawing to the actual screen. Skip this and nothing shows.
- `pygame.quit()` — shut down cleanly.

A Surface is a rectangle of pixels.

Everything you draw with pygame is drawn ONTO a Surface. The window is one. Images you load are too. You can create your own. Drawing a Surface onto another is called blitting.

```
# The screen IS a Surface — the one your OS shows.
screen = pygame.display.set_mode((640, 480))

# Make your own — a 100×100 off-screen rectangle.
sticker = pygame.Surface((100, 100))
sticker.fill((220, 60, 60))      # paint it red (RGB tuple)

# Paste sticker onto screen at position (50, 30).
screen.blit(sticker, (50, 30))

# You can blit Surfaces onto Surfaces, repeatedly. Layers.
```

Mental model

- Surface = 2D grid of colored pixels, sitting in RAM.
- blit = 'paste this Surface onto that Surface, at this (x, y)'.
- The screen is just the Surface that happens to be visible.
- Images, text, sprites — all just Surfaces under the hood.
- RGB tuples: (red, green, blue), each 0–255. (0,0,0) = black. (255,255,255) = white.

Every frame: input → update → draw.

A pygame program is one while loop that runs ~60 times per second. Each pass does three things in this order: read input, change the world, paint the new picture. This pattern is universal.

```
clock = pygame.time.Clock()
ball_x, ball_y = 320, 240
ball_vx = 4

while running:
    # 1. INPUT — what did the user do?
    for event in pygame.event.get():
        if event.type == pygame.QUIT: running = False

    # 2. UPDATE — change the world
    ball_x += ball_vx
    if ball_x > 640 or ball_x < 0: ball_vx = -ball_vx

    # 3. DRAW — paint the new state
    screen.fill((20, 20, 30))
    pygame.draw.circle(screen, (255, 255, 255), (ball_x, ball_y), 20)
    pygame.display.flip()
    clock.tick(60)           # hold 60 FPS
```

Why this order matters

- INPUT first — you want this frame's actions to affect this frame's update.
- UPDATE next — apply the inputs, advance physics, check rules.
- DRAW last — paint the world as it now is.
- If you draw before updating, you'll always render one frame behind.
- `screen.fill()` clears the last frame. Forget it and old frames stay visible → trails.
- `display.flip()` makes it visible. `clock.tick(60)` waits before next loop.

Reading user input.

When the user presses a key, clicks, or moves the mouse, pygame puts an Event object into a queue. Each frame, you drain the queue and react to whatever's relevant.

```
for event in pygame.event.get():  
  
    # Window-close button clicked  
    if event.type == pygame.QUIT:  
        running = False  
  
    # A key was just pressed  
    elif event.type == pygame.KEYDOWN:  
        if event.key == pygame.K_SPACE: paused = not paused  
        elif event.key == pygame.K_RIGHT: ball_vx += 1  
  
    # Mouse button clicked  
    elif event.type == pygame.MOUSEBUTTONDOWN:  
        print("clicked at", event.pos)
```

Key concepts

- event.type tells you WHAT kind (KEYDOWN, MOUSEBUTTONDOWN, QUIT, ...).
- event.key / event.pos give the DETAILS.
- KEYDOWN fires ONCE when key is first pressed.
- For 'is key currently held' use pygame.key.get_pressed() instead.
- Always drain event.get() every frame — even if you ignore most events.
- Skip it for ~1 sec and the OS marks your program as frozen.

Why `clock.tick(60)` matters.

Without a clock, your while loop runs as fast as the CPU can manage — thousands of frames per second. Everything you draw becomes instantly invisible to the human eye. `clock.tick(60)` puts a sane upper bound on speed.

```
clock = pygame.time.Clock()
```

while running:

```
# ... events, update, draw, flip ...
```

```
clock.tick(60)
```

```
# What tick(60) does internally:
```

```
# 1. Look at the time since the last tick() call.
```

```
# 2. If less than 1/60 sec has passed, sleep the difference.
```

```
# 3. If more than 1/60 sec has passed, return immediately.
```

```
# It returns ms-elapsed since last call — useful if you want dt:
```

```
dt_ms = clock.tick(60)
```

```
dt = dt_ms / 1000.0    # seconds
```

FPS choices

- 60 FPS — standard. Smooth, low input lag. Use this.
- 30 FPS — passable. Saves CPU on weak machines.
- 120 FPS — buttery, but most monitors only refresh at 60 anyway.
- Don't omit `tick()` — your laptop fan will scream and the ball flies off-screen in 0.1s.
- Most games hard-code a 'frame' as 1/60 sec. Simpler than variable `dt`.

+y goes DOWN. This trips everyone.

pygame's origin (0, 0) is the TOP-LEFT corner of the window. +x goes right (as expected). +y goes DOWN — the opposite of math class. Until this becomes reflex, things will move the wrong direction.



Window is a rectangle. Origin top-left. +x right. +y down.

Direction-of-motion cheatsheet

```
ball_y -= 1 # moves UP on screen
ball_y += 1 # moves DOWN on screen
ball_x -= 1 # moves LEFT on screen
ball_x += 1 # moves RIGHT on screen
```

- If your physics says 'up' but the object moves down — you forgot to flip a sign.
- Easiest workaround: write a `screen_y(world_y)` helper that negates internally.
- Once you accept the convention, it stops feeling wrong.

Rect, circle, line, polygon.

pygame.draw has four built-in shapes. Each one paints **DIRECTLY** onto a Surface — there's no scene graph or display list. You just say 'paint a red rectangle at (100, 50)' and it happens.

```
WHITE = (255, 255, 255)
RED   = (220, 60, 60)

# Rectangle — (surface, color, (x, y, width, height))
pygame.draw.rect(screen, RED, (100, 50, 80, 40))

# Circle — (surface, color, (cx, cy), radius)
pygame.draw.circle(screen, WHITE, (320, 240), 30)

# Line — (surface, color, start, end, thickness)
pygame.draw.line(screen, WHITE, (0, 100), (640, 100), 2)

# Polygon — (surface, color, [(x,y), (x,y), (x,y), ...])
pygame.draw.polygon(screen, RED, [(50,50), (100,50), (75,100)])
```

Argument patterns

- All four take Surface FIRST, then color, then geometry.
- Rect is (x, y, w, h) — top-left corner + size. NOT (x1, y1, x2, y2).
- Add an extra integer at the end → outline thickness. 0 / omitted = filled.
- Drawing is immediate — call happens, pixels change. No buffering inside `pygame.draw`.
- All shapes draw onto the surface you pass — usually screen, but could be any Surface.

Bouncing ball in a window.

One file: hello_pygame.py. White circle. Bounces off all four walls. 60 FPS. SPACE pauses.

SETUP

Window

- 640×480 window. Title: 'Day 8 — Bouncing Ball'.
- Background fill: dark navy (20, 20, 30).
- Initialize a Clock for 60 FPS.
- Set up ball position (start in center), velocity (4, 3 px/frame), radius (20).

LOOP

Three phases

- INPUT — handle QUIT and SPACE (toggle paused).
- UPDATE — if not paused: $\text{ball_x} += \text{vx}$, $\text{ball_y} += \text{vy}$.
- If ball crosses any wall, flip the matching velocity sign.
- DRAW — fill bg, draw circle at (ball_x, ball_y), flip().

EXTEND (optional)

If you finish early

- Trail effect — fill with a translucent bg instead of full clear.
- Use arrow keys to nudge the ball's velocity.
- Add 2 more balls. Use a list. Loop over it in update + draw.
- Change ball color when it hits a wall. Random RGB.

Goal isn't beauty — it's owning the loop. Once this runs smoothly, you understand 80% of pygame.

Things that will bite.

⚠ Nothing visible? You forgot `display.flip()`.

Drawing changes the back-buffer Surface in memory. `flip()` pushes that buffer to the actual screen. Without it, your screen never updates. Always last call in the loop body.

⚠ Trails everywhere? You forgot `screen.fill()`.

Each frame draws on top of the last. If you don't clear first, every previous frame stays visible. Call `screen.fill(bg_color)` at the start of the DRAW phase.

💡 Use integers for screen coordinates.

`pygame.draw` expects ints, not floats. If your position drifts from float arithmetic, wrap in `int()` at draw time: `(int(ball_x), int(ball_y))`. Keep the float in memory, cast for drawing.

💡 Check key state vs key event.

`pygame.KEYDOWN` fires once per press. `pygame.key.get_pressed()[K_LEFT]` returns True every frame the key is held. Use the first for toggles, the second for continuous movement.

Loop today → sprites tomorrow → Pong by Saturday.

TUE DAY 09 · Sprites

OOP version of today

Wrap that ball in a Sprite subclass. Multiple balls become a Group. `update()` and `draw()` get called on the whole group at once. Same loop, less repetition.

WED-FRI · Collisions, sound, scenes

Filling out the toolkit

Wed: rect/mask collisions for real shapes. Thu: `pygame.mixer` for sound + sprite animation. Fri: scene state machine so you can switch between menu, play, game-over.

SAT DAY 13 · Pong

Build it for real

Two paddles, a ball, scoring, a simple AI for the right paddle, a menu screen. Today's loop pattern unchanged. The ball is yours from today, the paddles are sprites from tomorrow.

Today is the slowest day of the week on purpose. Get this clean and the rest writes itself.